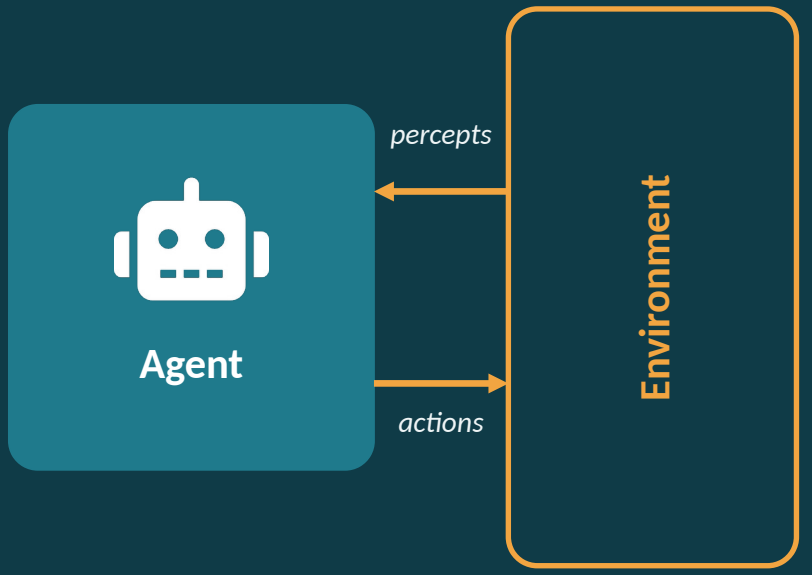
Intelligent Agents

How AI systems perceive, decide and act — and how we describe the worlds they act in



Based on Russell & Norvig, Artificial Intelligence: A Modern Approach (4th ed.), Chapter 2

Today's roadmap



01

Agents & environments

What an agent is; percepts, the agent function and the agent program



02

Rationality

Performance measures and what “doing the right thing” means



03

Task environments

The PEAS description and the seven properties of environments



04

Agent structure

From simple reflex agents to learning agents; how states are represented



Why start here? The agent view frames the whole course: search, reasoning under uncertainty and machine learning are all ways of building better agents.

01

Agents & Environments

Who acts, what they perceive, and what they can do

Agent | Percepts | Agent function | Agent program | Vacuum world

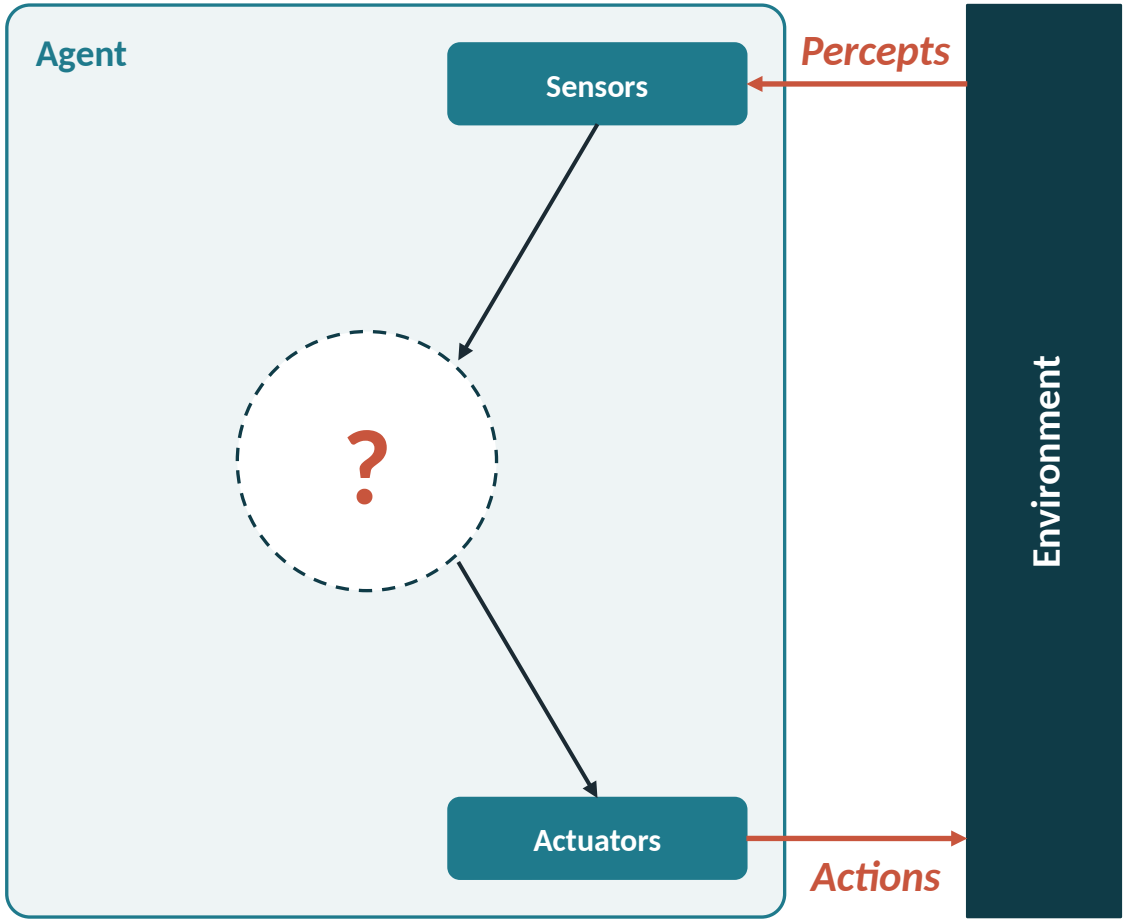
What is an agent?

DEFINITION · AGENT

An agent is anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.

Environment: in principle everything; in practice, the part of the world whose state we care about when designing the agent — what affects what the agent perceives and is affected by its actions.

Percepts flow **in** through sensors; actions flow **out** through actuators. Everything in this chapter builds on this loop.



Agents come in many forms

Human agent

SENSORS
Eyes, ears and other organs

ACTUATORS
Hands, legs, vocal tract

Robotic agent

SENSORS
Cameras, infrared range finders

ACTUATORS
Various motors

Software agent

SENSORS
File contents, network packets, keyboard / mouse / touchscreen / voice input

ACTUATORS
Writing files, sending network packets, displaying information, generating sounds

"Agent" is a lens, not a category. A calculator could be viewed as an agent that displays "4" after perceiving "2 + 2 =", but that view adds nothing. AI focuses on systems where decision making is non-trivial.

Percepts, the agent function and the agent program

Percept

The content an agent's sensors are perceiving at a given moment.

Percept sequence

The complete history of everything the agent has ever perceived.

Agent function

Maps any given percept sequence to an action. It fully describes the agent's behaviour.

Function vs. program

ABSTRACT

Agent function

- Mathematical description
- External view of behaviour
- Could, in principle, be written as a (huge) table

→

CONCRETE

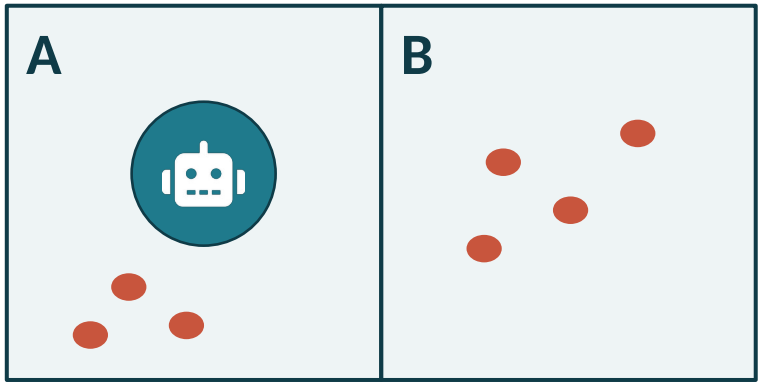
Agent program

- Implementation in code
- Runs on a physical system
- Takes only the current percept as input



An agent's choice of action can depend on its **built-in knowledge** and the **entire percept sequence** observed to date — but not on anything it hasn't perceived.

A running example: the vacuum-cleaner world



Percepts: [location, status], e.g. [A, Dirty]

Actions: Left, Right, Suck, NoOp

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

A SIMPLE AGENT FUNCTION

If the current square is **dirty** → **Suck**;
otherwise →
move to the other square.

02

Rationality

Good behaviour: what does it mean to “do the right thing” ?

Performance measure | Rational agent | Omniscience | Learning | Autonomy

Performance measures

DEFINITION · PERFORMANCE MEASURE

A **performance measure** evaluates any given sequence of environment states. AI generally follows **consequentialism**: we judge an agent's behaviour by its consequences.



A flawed measure

Amount of dirt cleaned up in one eight-hour shift.

A rational agent can clean up the dirt, dump it back on the floor, and clean it again — maximising the score.



A better measure

One point for each clean square at each time step (perhaps with a penalty for electricity used and noise made).

Rewards the outcome we actually want: a clean floor.



Rule of thumb: design performance measures according to what you actually want to achieve in the environment, not according to how you think the agent should behave.

The rational agent

DEFINITION · RATIONAL AGENT

For each possible percept sequence, a **rational agent** should select an action that is **expected to maximize its performance measure**, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

What is rational at any given time depends on four things



Performance measure

The criterion of success



Prior knowledge

What the agent knows about the environment in advance



Available actions

What the agent is able to do



Percept sequence

Everything the agent has perceived so far

Is the vacuum agent rational? It depends

Under these assumptions…

- Performance: +1 per clean square per time step, over 1000 steps
- Geography known; dirt locations and start square not known
- Actions: Left, Right, Suck (clean squares stay clean)
- Agent perceives its location and whether there is dirt

Rational: no other agent does better in expectation

…but change the setting and it is not

Penalty of −1 per move

Oscillating once everything is clean wastes points → do nothing when sure all is clean.

Clean squares can get dirty again

Should occasionally check and re-clean squares.

Geography is unknown

Must explore the environment rather than assume two squares.

Rationality is always **relative** to the performance measure, prior knowledge, available actions and percepts.

Rational is not omniscient

Rational ≠ perfect

Rationality maximises expected performance; perfection maximises actual performance. Crossing an empty street is rational — even if a falling cargo door then flattens you.

Information gathering

Taking actions to improve future percepts is part of being rational: look both ways before crossing; explore an unknown environment.

Learning

A rational agent learns as much as possible from what it perceives. Without learning, agents are fragile: the dung beetle keeps “plugging” its nest even after its dung ball is removed.

DEFINITION · AUTONOMY

An agent that relies on its designer's prior knowledge rather than on its own percepts and learning **lacks autonomy**. A rational agent should be **autonomous**: it should learn what it can to compensate for partial or incorrect prior knowledge.

03

Task Environments

The “problems” to which rational agents are the “solutions”

PEAS | Observable | Agents | Deterministic | Episodic | Static | Discrete | Known

Specifying the task environment: PEAS

DEFINITION · TASK ENVIRONMENT

The **task environment** is the problem a rational agent must solve. We specify it with a **PEAS** description: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors.

P

Performance measure

How do we judge success?

E

Environment

Where does the agent operate, and with what?

A

Actuators

How does the agent act on the world?

S

Sensors

How does the agent perceive the world?



In designing an agent, the first step must always be to **specify the task environment as fully as possible**.

PEAS example: an automated taxi

P Performance

- Safe, fast, legal, comfortable trip
- Maximise profits
- Minimise impact on other road users

E Environment

- Roads (rural lanes to freeways)
- Other traffic, pedestrians, police
- Customers
- Weather

A Actuators

- Steering, accelerator, brake
- Signal, horn
- Display, speech

S Sensors

- Cameras, radar, lidar
- Speedometer, accelerometer, GPS
- Engine sensors
- Microphones, touchscreen



Trade-offs: goals conflict (speed vs. safety vs. profit). The more restricted the environment, the easier the design problem.

More PEAS examples

Agent type	Performance measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments	Touchscreen / voice entry of symptoms and findings
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, tactile and joint-angle sensors
Refinery controller	Purity, yield, safety	Refinery, raw materials, operators	Valves, pumps, heaters, stirrers, displays	Temperature, pressure, flow, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, feedback, speech	Keyboard entry, voice
Online product recommender	Click-through, purchases, customer satisfaction	Web shop, customers, product catalogue	Show recommendations, send emails / notifications	Clicks, browsing history, purchases, ratings

Seven ways task environments differ

Fully vs. partially observable

Do the sensors give access to the complete relevant state?

Single-agent vs. multiagent

Are there other agents whose goals depend on my behaviour?

Deterministic vs. nondeterministic

Is the next state fully determined by the current state and action?

Episodic vs. sequential

Does the current decision affect future decisions?

Static vs. dynamic

Can the world change while the agent is deliberating?

Discrete vs. continuous

Are states, time, percepts and actions discrete or continuous?

Known vs. unknown

Does the agent know the “laws of physics” of the environment?

THE HARDEST CASE

Partially observable, multiagent, nondeterministic, sequential, dynamic, continuous and unknown

Taxi driving is hard in all these senses — except that its environment is mostly known.

Fully vs. partially observable



Fully observable

The agent's sensors give it access to the complete state of the environment at each point in time — or at least every aspect relevant to the choice of action.



Partially observable

Noisy or inaccurate sensors, or parts of the state are simply missing from the sensor data.



Unobservable

The agent has no sensors at all. Its goals may still be achievable, sometimes with certainty.

Why it matters

In a fully observable environment the agent need not keep any internal state to track the world. Partial observability forces it to remember.

EXAMPLES

- Chess board: fully observable
- Vacuum with only a local dirt sensor: cannot see dirt in other squares
- Taxi: cannot see what other drivers are thinking

Single-agent vs. multiagent

Single-agent

Solving a crossword puzzle alone

Multiagent

Playing chess: a two-agent environment

KEY TEST

Treat another entity B as an **agent** if B's behaviour is best described as **maximising a performance measure whose value depends on agent A's behaviour**.

Competitive

Chess: the opponent maximises its measure, which minimises yours.

Partially cooperative

Taxi driving: avoiding collisions benefits all drivers…

…and partially competitive

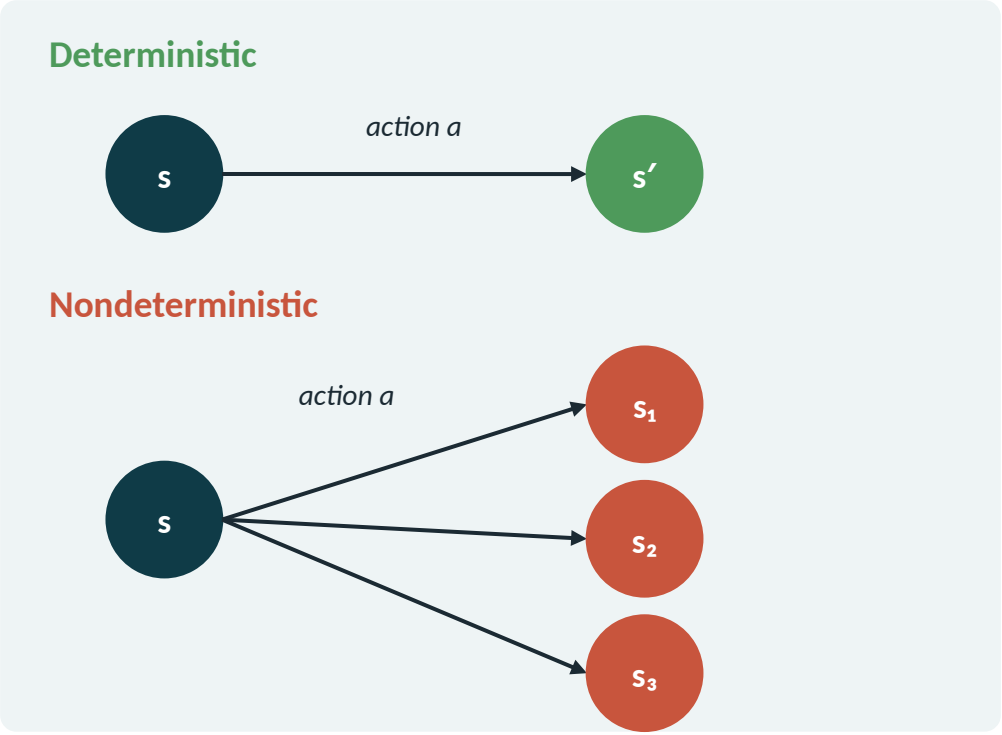
…but only one car can occupy a parking space.

Design consequences: communication can emerge as rational behaviour; in competitive settings, randomised behaviour can be rational because it avoids predictability.

Deterministic vs. nondeterministic

DEFINITION · DETERMINISTIC

If the next state of the environment is **completely determined** by the current state and the action executed by the agent(s), the environment is deterministic; otherwise it is **nondeterministic**.



- Vacuum world (as described): deterministic
- Taxi driving: nondeterministic — traffic is unpredictable, tyres blow out, engines seize
- A partially observable environment can appear nondeterministic

Stochastic: possibilities have explicit probabilities — "25% chance of rain tomorrow"

Nondeterministic: possibilities are listed but not quantified — "a chance of rain tomorrow"

Episodic vs. sequential

EPISODIC

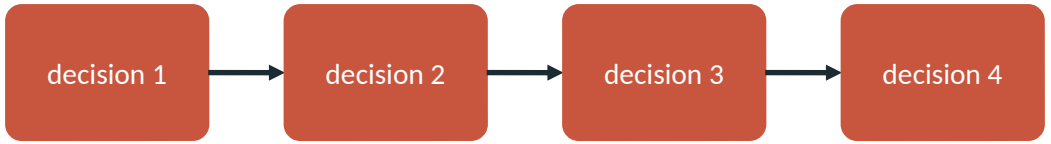
The agent's experience is divided into atomic **episodes**: receive a percept, perform one action. The next episode **does not depend** on actions taken in previous ones.



Examples: spotting defective parts on an assembly line; classifying images; filtering spam

SEQUENTIAL

The current decision **could affect all future decisions**. Short-term actions can have long-term consequences.



Examples: chess; taxi driving; a customer-service conversation

Episodic environments are much simpler: the agent **does not need to think ahead**.

Static vs. dynamic · Discrete vs. continuous

Static vs. dynamic

Static The environment does not change while the agent is deliberating.

e.g. crossword puzzles

Dynamic The environment keeps changing; the agent must keep looking at the world.

e.g. taxi driving

Semidynamic The environment does not change, but the agent's performance score does.

e.g. chess played with a clock

Discrete vs. continuous

Applies to the state of the environment, the way time is handled, and the percepts and actions.

Discrete A finite number of distinct states; discrete percepts and actions.

e.g. chess (ignoring the clock)

Continuous Values sweep smoothly through a range over time.

e.g. taxi: speed, location, steering angle

Known vs. unknown

DEFINITION

This refers to the agent's (or designer's) knowledge of the environment's **"laws of physics"**. In a **known** environment, the outcomes (or outcome probabilities) of all actions are given. In an **unknown** one, the agent must learn how it works.

	Fully observable	Partially observable
Known	Chess: rules known, whole board visible	Solitaire: rules known, but unturned cards are hidden
Unknown	New video game: whole screen visible, but what do the buttons do?	Driving a rented car in a new country with unfamiliar roads and laws

The performance measure can be unknown too

A taxi may not know whether a new passenger prefers a leisurely or a speedy trip; a virtual assistant starts knowing nothing about its owner's preferences. The agent must learn them through interaction.

Classifying familiar task environments

Task environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Shaded = the harder value of each property. Taxi driving is the hardest row; the crossword the easiest.

04

The Structure of Agents

agent = architecture + program

Simple reflex | Model-based | Goal-based | Utility-based | Learning | Representations

Agent programs and why lookup tables fail

- **agent** = **architecture** + **program**
- *architecture = device + sensors + actuators*
- *Program = implements the agent function*

TABLE-DRIVEN-AGENT (Figure 2.7)

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
              table, a table of actions, indexed by
              percept sequences, fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

$\geq 10^{150}$

entries in a lookup table for chess

$< 10^{80}$

atoms in the observable universe

Doomed: no room to store the table, no time to build it, and no way to learn every entry from experience.



The key challenge for AI: produce rational behaviour from a smallish program rather than from a vast table.

Simple reflex agents

DEFINITION

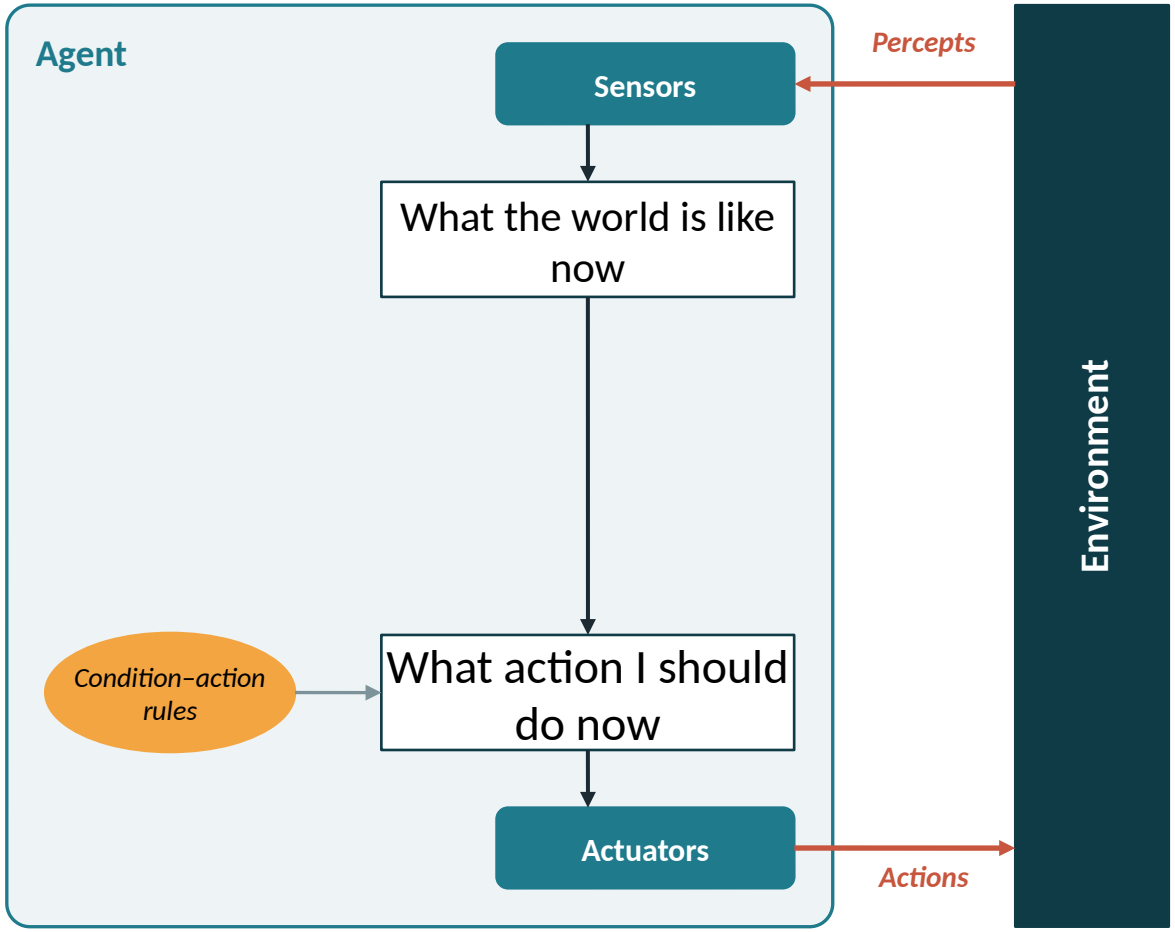
A **simple reflex agent** selects actions on the basis of the **current percept**, ignoring the rest of the percept history.

CONDITION–ACTION RULE

```
if car-in-front-is-braking then initiate-braking
```

```
function REFLEX-VACUUM-AGENT([location, status])
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

4 possible percepts instead of 4^T percept sequences



Where simple reflex agents break down

Needs full observability

It works only if the correct decision can be made from the current percept alone. A single video frame may not show whether the car ahead is braking or just has its tail-lights on.

Infinite loops

A vacuum agent with only a dirt sensor perceives just [Dirty] or [Clean]. If it always moves Left on [Clean], it fails forever when it starts in square A.

Escape by randomising

On [Clean], flip a coin between Left and Right: the agent reaches the other square in two steps on average — and can finish the job.



The real fix for partial observability: remember what you cannot currently see → model-based agents.

Model-based reflex agents

INTERNAL STATE

The agent keeps track of the part of the world it can't see now, maintaining an **internal state** that depends on the percept history.

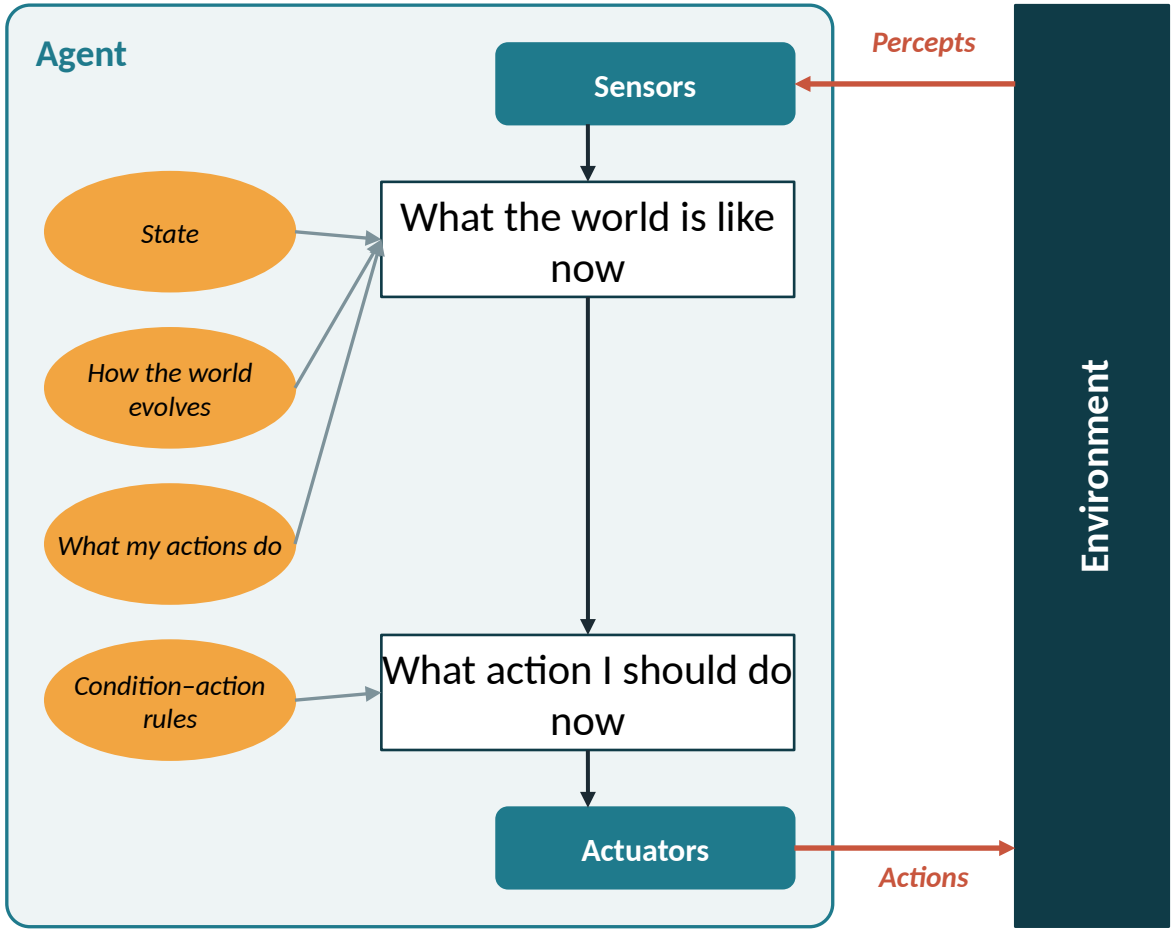
Transition model

How the world changes: the effects of the agent's actions and how the world evolves on its own. (Turn the wheel → the car turns.)

Sensor model

How the state of the world shows up in percepts. (Car ahead brakes → red regions appear in the camera image.)

State is a best guess: *certainty about the current state is rarely possible.*



Goal-based agents

GOAL

Information describing **desirable situations**, e.g. being at a particular destination. The agent combines goals with its model to choose actions that achieve them.

Considers the future

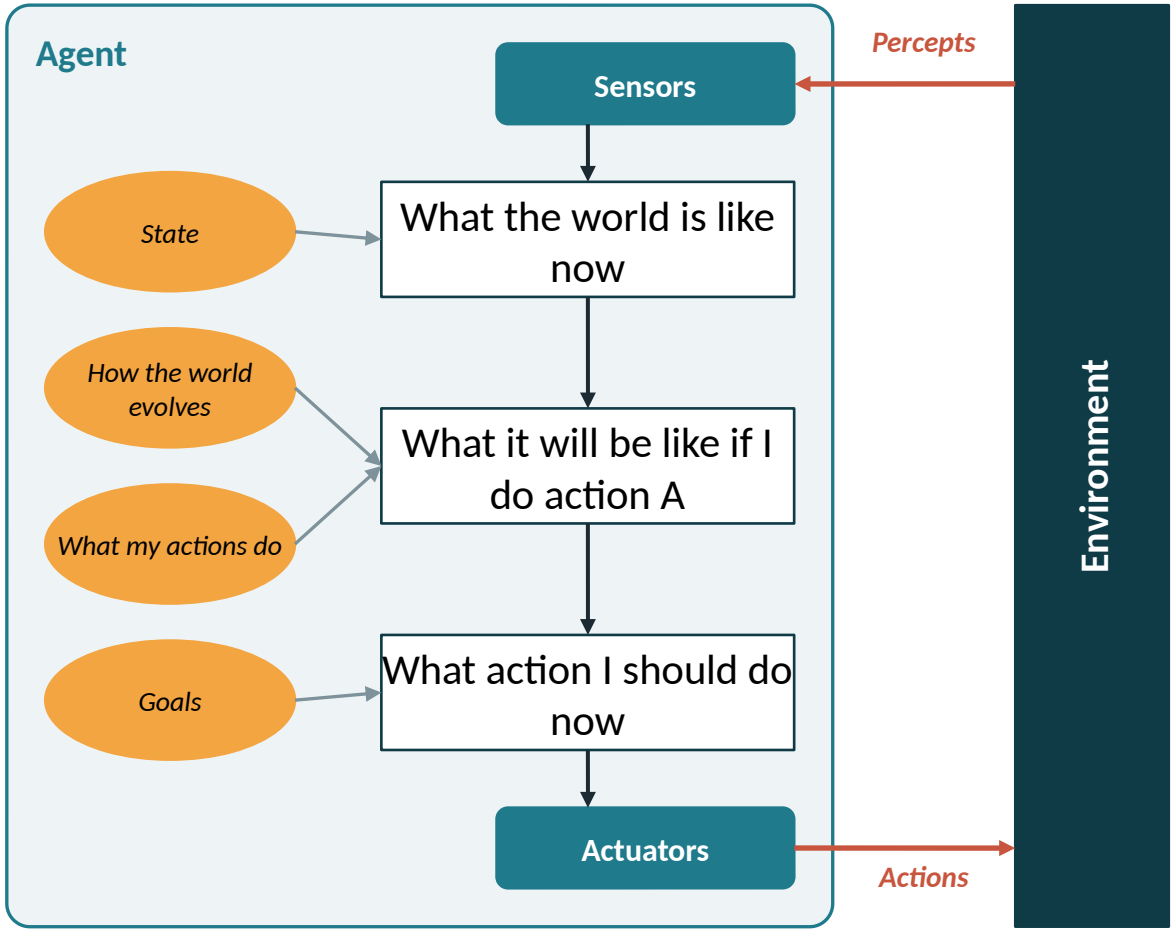
"What will happen if I do this?" and "Will that achieve my goal?"

Search and planning

Find action sequences that reach the goal — our next topic.

More flexible

Knowledge is explicit: to go elsewhere, change the goal, not every rule.



Utility-based agents

UTILITY FUNCTION

An internalisation of the performance measure: it scores **how desirable** each world state is. A rational utility-based agent chooses the action that **maximises expected utility**.

Degrees, not just yes/no

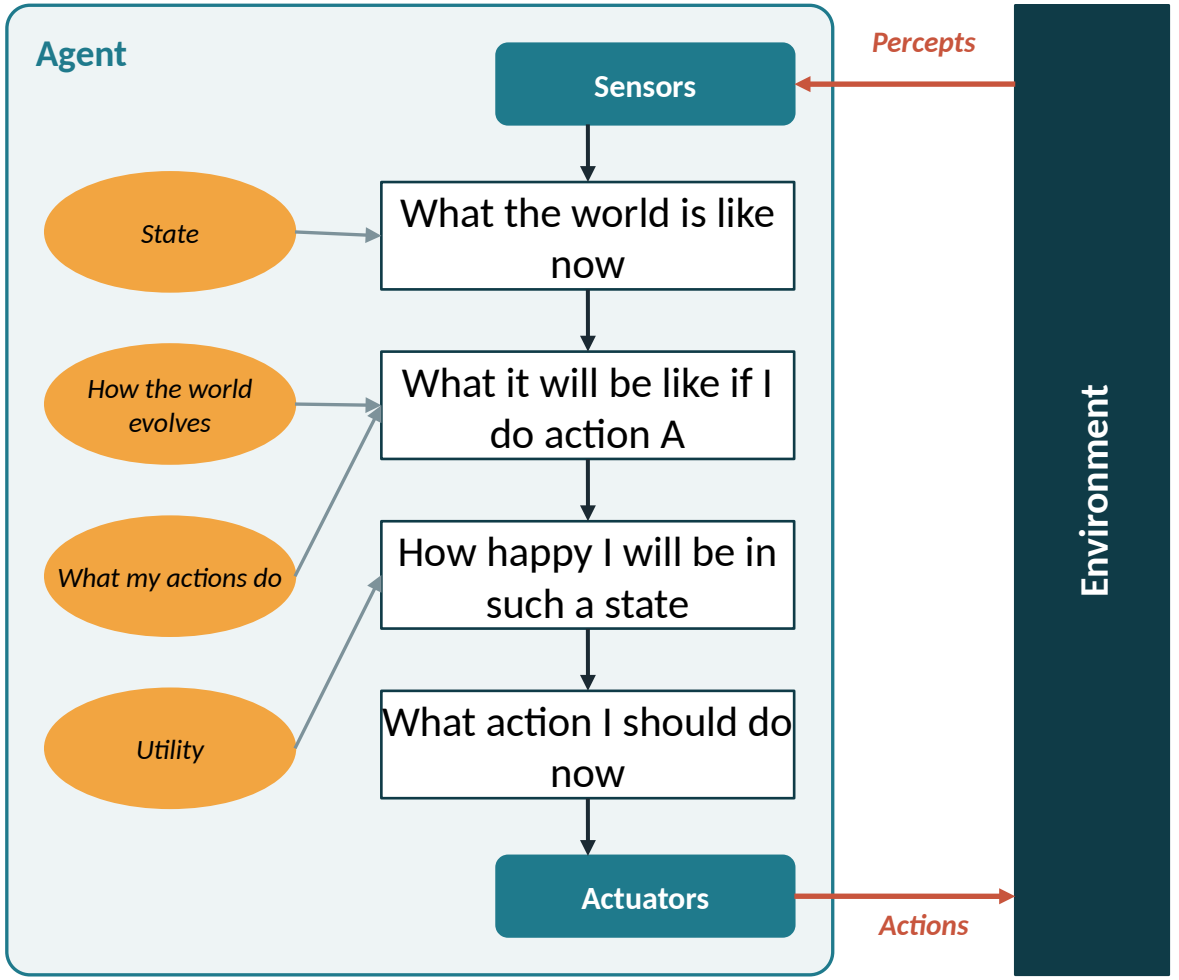
Goals only separate “happy” from “unhappy”. Utility can rank routes that are quicker, safer or cheaper.

Conflicting goals

Speed vs. safety: the utility function specifies the trade-off.

Uncertain goals

Weighs the likelihood of success against the importance of each goal.



Learning agents

Performance element

Selects external actions — what we previously called the whole agent.

Learning element

Makes improvements to the performance element, using feedback.

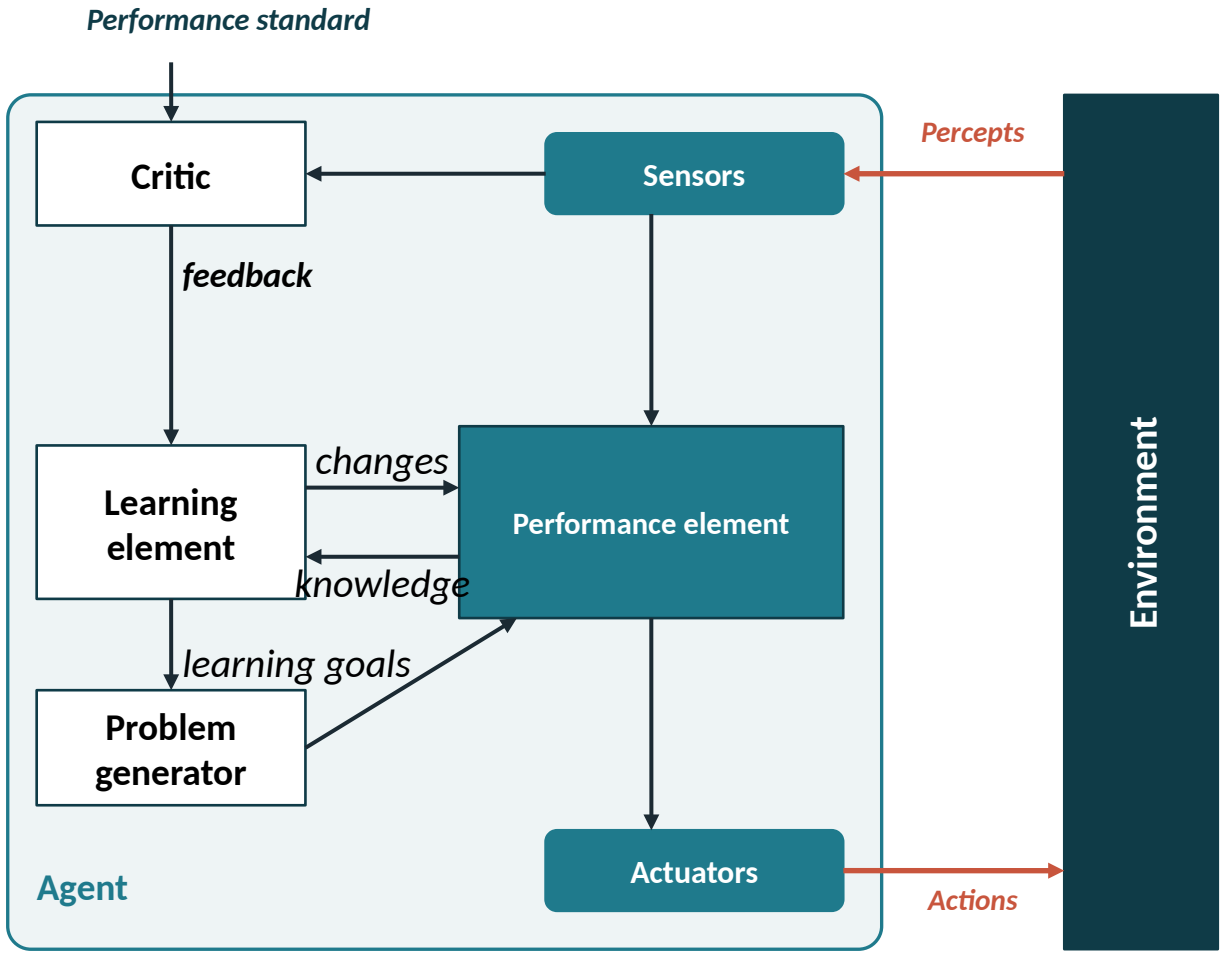
Critic

Tells the learning element how well the agent is doing against a fixed performance standard.

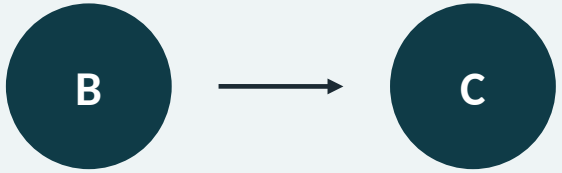
Problem generator

Suggests exploratory actions that lead to new, informative experiences.

Learning = adjusting each component to agree better with feedback.



How agents represent the world

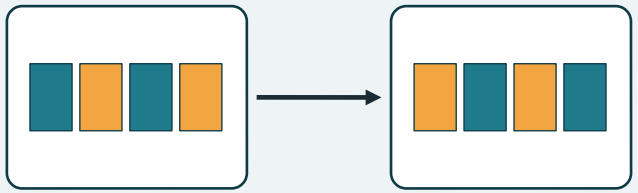


Atomic

Each state is indivisible — a black box with no internal structure.

Route finding: the state is just the name of the city.

Used in: Search, game playing, hidden Markov models, MDPs

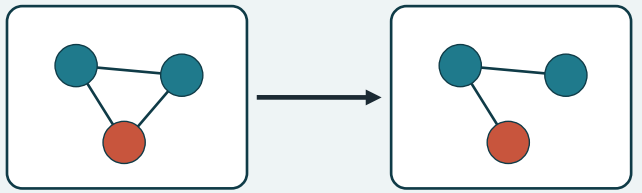


Factored

A state is a fixed set of variables (attributes), each with a value.

Driving: fuel level, GPS position, money for tolls, oil light…

Used in: Constraint satisfaction, logic, planning, Bayesian networks, most ML — every row of a Pandas table



Structured

A state contains objects with attributes and relationships between them.

A truck reversing into a dairy-farm driveway, blocked by a loose cow.

Used in: Relational databases, first-order logic, natural-language understanding

Increasing expressiveness → more concise descriptions, but reasoning and learning become harder

Key takeaways

1 Agents

An agent perceives through sensors and acts through actuators; the agent function maps percept sequences to

2 Rationality

A rational agent maximises expected performance, given its percepts and built-in knowledge — not perfection.

3 PEAS first

Specify the task environment — performance, environment, actuators, sensors — before designing the agent.

4 Environments vary

Observable, agents, deterministic, episodic, static, discrete, known: these decide which techniques fit.

5 Agent programs

Simple reflex → model-based → goal-based → utility-based; any of them can be a learning agent.

6 Representations

States can be atomic, factored or structured — trading expressiveness against ease of reasoning.

Next: solving problems by search — goal-based agents that plan sequences of actions.